For Docker:

deployment.yaml :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: static-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: static-web
  template:
    metadata:
      labels:
        app: static-web
    spec:
      containers:
      - name: static-web
        image: vignesh201005/static-web:latest
```

---

service.yaml:

```yaml
apiVersion: v1
kind: Service
```

```
metadata:
  name: static-web-service
spec:
  type: NodePort
  selector:
    app: static-web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30025
```

…….

Dockerfile :

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

……

One html file based on the question

```
kubectl apply -f deployment.yaml
```

……..

```
kubectl apply -f service.yaml
```

……..

```
kubectl get pods
```

……..

```
kubectl port-forward service/static-web-service 8080:80
```

……

Pipe line script :

```
pipeline {
    agent any
```

```
environment {
    DOCKER_IMAGE = "vignesh201005/static-web:latest"
}

stages {

    stage('Clone Repo') {
        steps {
            git 'https://github.com/Vignesh-2010/dockerdemo.git'
        }
    }

    stage('Build Docker Image') {
        steps {
            sh 'docker build -t $DOCKER_IMAGE .'
        }
    }

    stage('Login to DockerHub') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'dockerhub',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )]) {
                sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
            }
        }
    }
```

```
        stage('Push Docker Image') {
            steps {
                sh 'docker push $DOCKER_IMAGE'
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
                    sh 'kubectl apply -f deployment.yaml'
                    sh 'kubectl apply -f service.yaml'
                }
            }
        }
    }

    post {
        success {
            echo 'Pipeline executed successfully ✅'
        }
        failure {
            echo 'Pipeline failed ❌'
        }
    }
}
```

……

MAVEN :

Jenkins setting tool what ever that name(Maven or M3) is there that should be kept Pipeline script

And git hub url should be changed

In Pom.xml , **<artifactId>**(here givev outer folder name in vs code) **</artifactId>**

……..

Alright — forget the specific scenario (student system / restaurant page). Your exam is testing **process + DevOps workflow**, not the exact application. I'll give you a **universal step-by-step method** you can follow for ANY similar question.

✅ PART 1: MAVEN PROJECT + GITHUB + JENKINS PIPELINE

This applies to:
👉 Student Performance System
👉 Any Java backend app

🔹 STEP 1: Create GitHub Repository

1. Go to GitHub
2. Click **New Repository**
3. Name it:
4. student-performance-system
5. Select:
    - ✅ Public
    - ✅ Add README
6. Click **Create Repository**

🔹 STEP 2: Create Maven Project (Standard Structure)

In VS Code Terminal:

mvn archetype:generate

Provide:

GroupId: com.example

ArtifactId: student-app

This creates structure:

```
student-app/
 ├── src/
 |   ├── main/java/
```

```
|   ├── test/java/
 ├── pom.xml
```

🔹 **STEP 3: Write Basic Java Logic**

Go to:

src/main/java/com/example/App.java

Example (works for ANY scenario — modify logic only):

```java
import java.util.*;

public class App {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int students = 5;

        for(int i=0;i<students;i++){
            System.out.println("Enter CAT, Assignment, FAT:");
            int cat = sc.nextInt();
            int assign = sc.nextInt();
            int fat = sc.nextInt();

            int total = cat + assign + fat;
            double gpa = total / 10.0;

            System.out.println("Total: " + total);
            System.out.println("GPA: " + gpa);
        }
    }
}
```

👉 For different questions → just change logic, **structure stays same**

🔹 **STEP 4: Connect GitHub to VS Code**

```
git init
git remote add origin https://github.com/your-repo-link.git
git add .
git commit -m "Initial commit"
git push -u origin master
```

🔹 **STEP 5: Clone Repo (Exam Step)**

```
git clone https://github.com/your-repo-link.git
cd student-performance-system
```

Open in VS Code:

```
code .
```

🔹 **STEP 6: Modify Code + Commit**

Make changes → then:

```
git add .
git commit -m "Updated logic"
git push
```

🔹 **STEP 7: Build Project (IMPORTANT for Jenkins)**

```
mvn clean install
```

If build SUCCESS → ✅ ready for CI/CD

🔹 **STEP 8: Setup Jenkins Pipeline**

Open Jenkins

Create Pipeline Job:

1. New Item
2. Enter name
3. Select **Pipeline**

🔹 **STEP 9: Pipeline Script**

Paste this:

```
pipeline {
  agent any

  stages {
    stage('Clone') {
      steps {
        git 'https://github.com/your-repo-link.git'
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean install'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
    }
  }
}
```

Click:

Build Now

✔ DONE PART 1

✅ **PART 2: HTML + DOCKER + JENKINS + KUBERNETES**

This applies to:
👉 Restaurant Website
👉 Any static web page

🔹 **STEP 1: Create HTML Page**

Create index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Restaurant Menu</title>
</head>
<body>

<h1>My Restaurant</h1>
<p>Delicious Food Everyday</p>

<h2>Starters</h2>
<ul>
  <li>Soup - ₹100</li>
</ul>

<h2>Main Course</h2>
<ul>
  <li>Biryani - ₹200</li>
</ul>

<h2>Desserts</h2>
<ul>
  <li>Ice Cream - ₹80</li>
</ul>
```

```
<h2>Beverages</h2>
<ul>
  <li>Juice - ₹60</li>
</ul>

<footer>
  Contact: 9876543210
</footer>

</body>
</html>
```

🔹 **STEP 2: Push to GitHub**

Same steps:

```
git init
git add .
git commit -m "menu page"
git push
```

🔹 **STEP 3: Dockerize Application**

Create Dockerfile:

```
FROM nginx:latest
COPY . /usr/share/nginx/html
```

🔹 **STEP 4: Build Docker Image**

Using Docker:

```
docker build -t menu-app .
```

Run:

```
docker run -p 8080:80 menu-app
```

Open browser:

http://localhost:8080

🔹 STEP 5: Kubernetes Deployment

Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: menu-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: menu
  template:
    metadata:
      labels:
        app: menu
    spec:
      containers:
      - name: menu-container
        image: menu-app
        ports:
        - containerPort: 80
```

🔹 STEP 6: Service File

```
apiVersion: v1
kind: Service
metadata:
  name: menu-service
spec:
  type: NodePort
  selector:
```

```
    app: menu
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Apply:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

🔹 **STEP 7: Jenkins Pipeline (Docker + K8s)**

In Jenkins:

```
pipeline {
    agent any

    stages {
        stage('Clone') {
            steps {
                git 'https://github.com/your-repo-link.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t menu-app .'
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
                sh 'kubectl apply -f service.yaml'
```

```
        }
      }
    }
}
```

🎯 **FINAL EXAM STRATEGY (IMPORTANT)**

No matter the scenario:

✔️ **ALWAYS DO THIS FLOW:**

🔹 **For Java/Maven Question:**

1. GitHub repo
2. Maven project
3. Write logic
4. Push code
5. Clone in VS Code
6. Modify + commit
7. mvn clean install
8. Jenkins pipeline

🔹 **For Web/Docker Question:**

1. Create HTML page
2. Push to GitHub
3. Clone + modify
4. Dockerfile
5. Build image
6. Run container
7. Kubernetes deploy
8. Jenkins pipeline

⚡ **LAST MINUTE TIPS**

- If stuck → **write structure + commands → you'll get marks**
- Even if code not perfect → pipeline steps matter more

- Always include:
    - git add .
    - git commit
    - git push
    - mvn clean install
    - docker build